UNIT - 03

HOSPITAL PATIENT MANAGEMENT SYSTEM

Problem Statement:

Develop a hospital patient management system using a doubly linked list, a stack and an array. Store patient details such as patient ID, name, age, disease and allotted bed number in a doubly linked list. The system should allow admitting new patients, discharging patients, searching for a patient and displaying all patient records in forward and backward order. Hospital beds are managed using an array for bed allocation, and discharged patients are saved in a stack so that the last discharge can be undone.

Objective:

The objective of this project is to develop a hospital patient management system in C++ using structures without classes. Patient details such as ID, name, age, disease and allocated bed number are stored in a doubly linked list, which allows easy insertion, deletion, searching and traversal in both forward and backward direction. Hospital beds are managed using an array, so the system can allot a free bed to each new patient. Discharged patients are stored in a stack, which keeps the discharge history and lets the last discharge be undone. Through this program we apply linked list, stack and array concepts to solve real-world hospital problems efficiently.

Algorithm:

Step 1: Start. Create an empty doubly linked list, an array of 10 free beds and an empty stack.

Step 2: Display the menu: admit, discharge, search, display forward, display backward, bed status, undo discharge, history, exit.

Step 3: Read the choice and call the matching function.

Step 4: Repeat Steps 2 and 3 until choice = 0.

Step 5: Admit: Find the first free bed in the array. If none is free, display "No beds available" and stop.

Step 6: Read patient ID. If the ID already exists in the list, stop.

Step 7: Read name, age and disease.

Step 8: Create a new node and insert it at the end of the list (update tail and prev/next links).

Step 9: Mark the bed as occupied and increase count.

Step 10: Discharge: Read patient ID and search the list from head. If not found, display "Patient not found" and stop.

Step 11: Push the patient record onto the stack (top = top + 1).

Step 12: Free the patient's bed in the array.

Step 13: Delete the node by linking its previous and next nodes (update head or tail if needed).

Step 14: Decrease count.

Step 15: Search: Read the patient ID or name.

Step 16: Traverse the list from head using next until NULL.

Step 17: If a node matches, display the patient details; otherwise display "Patient not found".

Step 18: Forward: Start at head and print each node using next until NULL.

Step 19: Backward: Start at tail and print each node using prev until NULL.

Step 20: Undo discharge: If the stack is empty, display a message and stop.

Step 21: Pop the top record (top = top - 1).

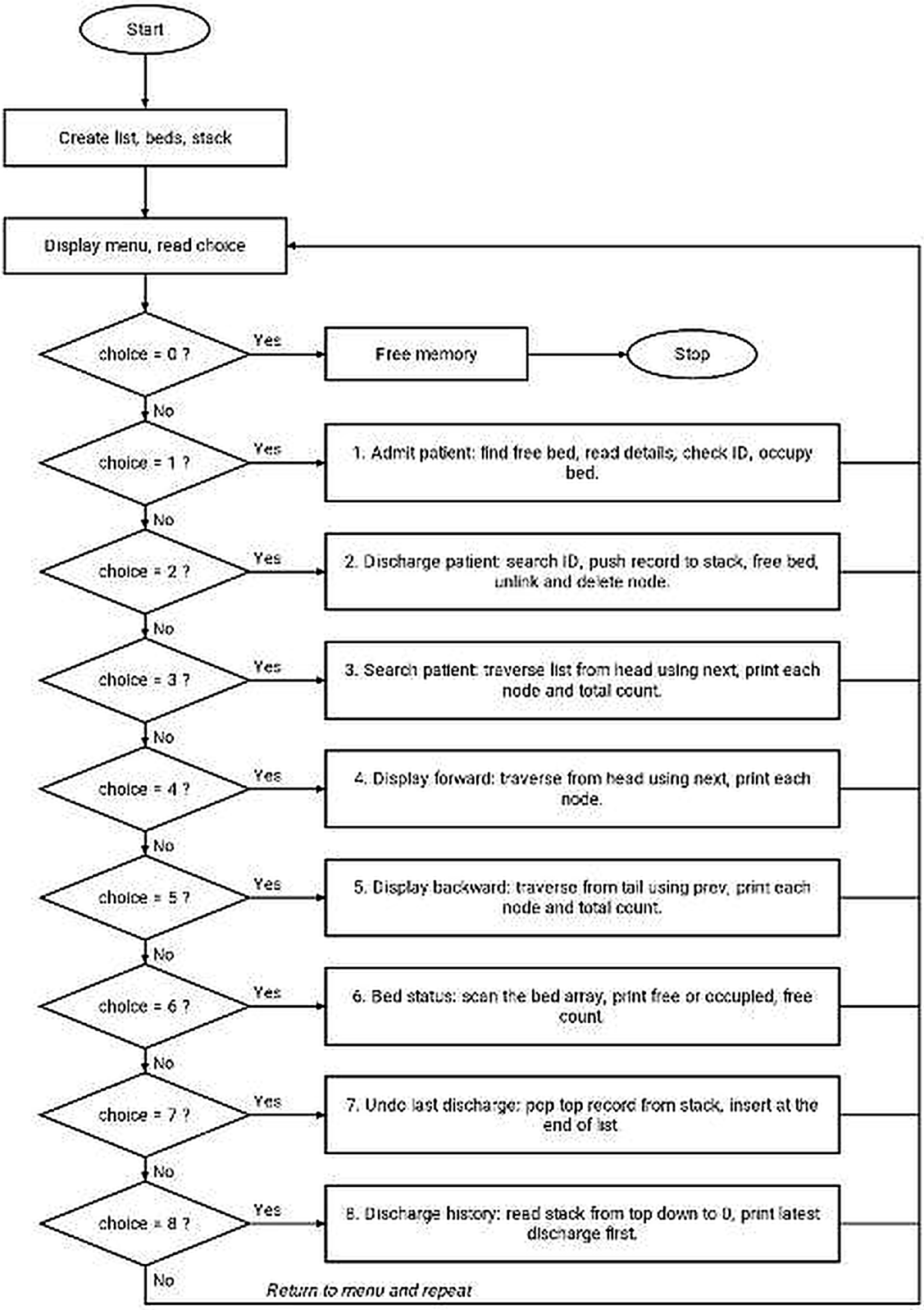
Step 22: Insert the patient at the end of the list and mark a free bed as occupied.

Step 23: Bed status: Traverse the bed array and print each bed as free or occupied.

Step 24: History: Print the stack records from top to 0 (latest discharge first).

Step 25: Stop.

Flowchart:


Start
Create list, beds, stack
Display menu, read choice
choice = 0 ?
Yes
Free memory
Stop
No
choice = 1 ?
Yes
1. Admit patient: find free bed, read details, check ID, occupy bed.
No
choice = 2 ?
Yes
2. Discharge patient: search ID, push record to stack, free bed, unlink and delete node.
No
choice = 3 ?
Yes
3. Search patient: traverse list from head using next, print each node and total count.
No
choice = 4 ?
Yes
4. Display forward: traverse from head using next, print each node.
No
choice = 5 ?
Yes
5. Display backward: traverse from tail using prev, print each node and total count.
No
choice = 6 ?
Yes
6. Bed status: scan the bed array, print free or occupied, free count
No
choice = 7 ?
Yes
7. Undo last discharge: pop top record from stack, insert at the end of list
No
choice = 8 ?
Yes
8. Discharge history: read stack from top down to 0, print latest discharge first.
No
Return to menu and repeat